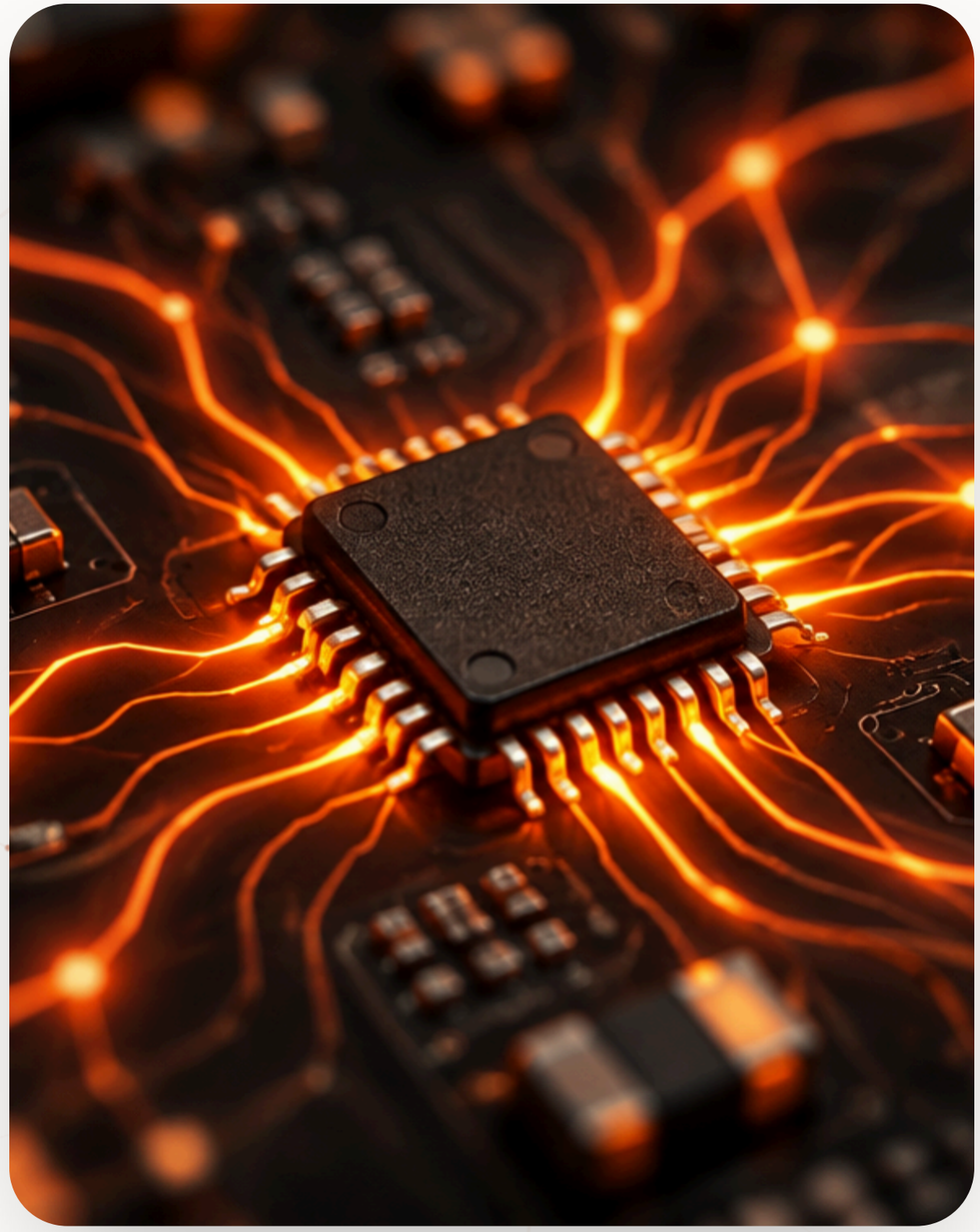
UNIVERSITÉ LE HAVRE NORMANDIE

MODULE INTERNET DES OBJETS · VEILLE DOCUMENTAIRE

L'AIoT & le TinyML

Quand l'intelligence artificielle s'intègre dans les objets connectés



Présenté par

Hamadou BA · Nicolas CHEVALIER · Mohamed SANOGO

Encadrant

M. Claude DUVALLET

Plan

HB

Hamadou BA

Partie 1

De l'IoT à l'AIoT et au TinyML

- L'objet connecté et l'architecture IoT
- Les limites du « tout cloud »
- AIoT et TinyML : définitions, contraintes
- Cloud ou inférence locale ?

NC

Nicolas CHEVALIER

Parties 2 et 3

Principes techniques, matériel et outils

- Le pipeline de développement
- Quantification, élagage, distillation
- ESP32, STM32N6 : le matériel
- TFLM, Edge Impulse : les outils

MS

Mohamed SANOGO

Parties 4 et 5

Bénéfices, applications et limites

- Réactivité, confidentialité, énergie
- Exemple : surveillance d'un moteur
- Limites, sécurité, perspectives
- Conclusion

LE CONTEXTE

18,8 milliards

d'objets connectés actifs dans le monde fin 2024 (IoT Analytics)

Ils mesurent

Température, vibrations, son, mouvement…

Ils transmettent

Des flux continus de données brutes vers le cloud.

Et s'ils décidaient ?

Analyser **dans l'objet**, n'envoyer que l'essentiel.

PROBLÉMATIQUE

Comment exécuter une IA conçue pour des serveurs sur un objet qui ne dispose que de quelques centaines de kilooctets, et qu'y gagne-t-on ?

01
IoT, AIoT & TinyML
Hamadou

02
Principes techniques
Nicolas

03
Matériel & outils
Nicolas

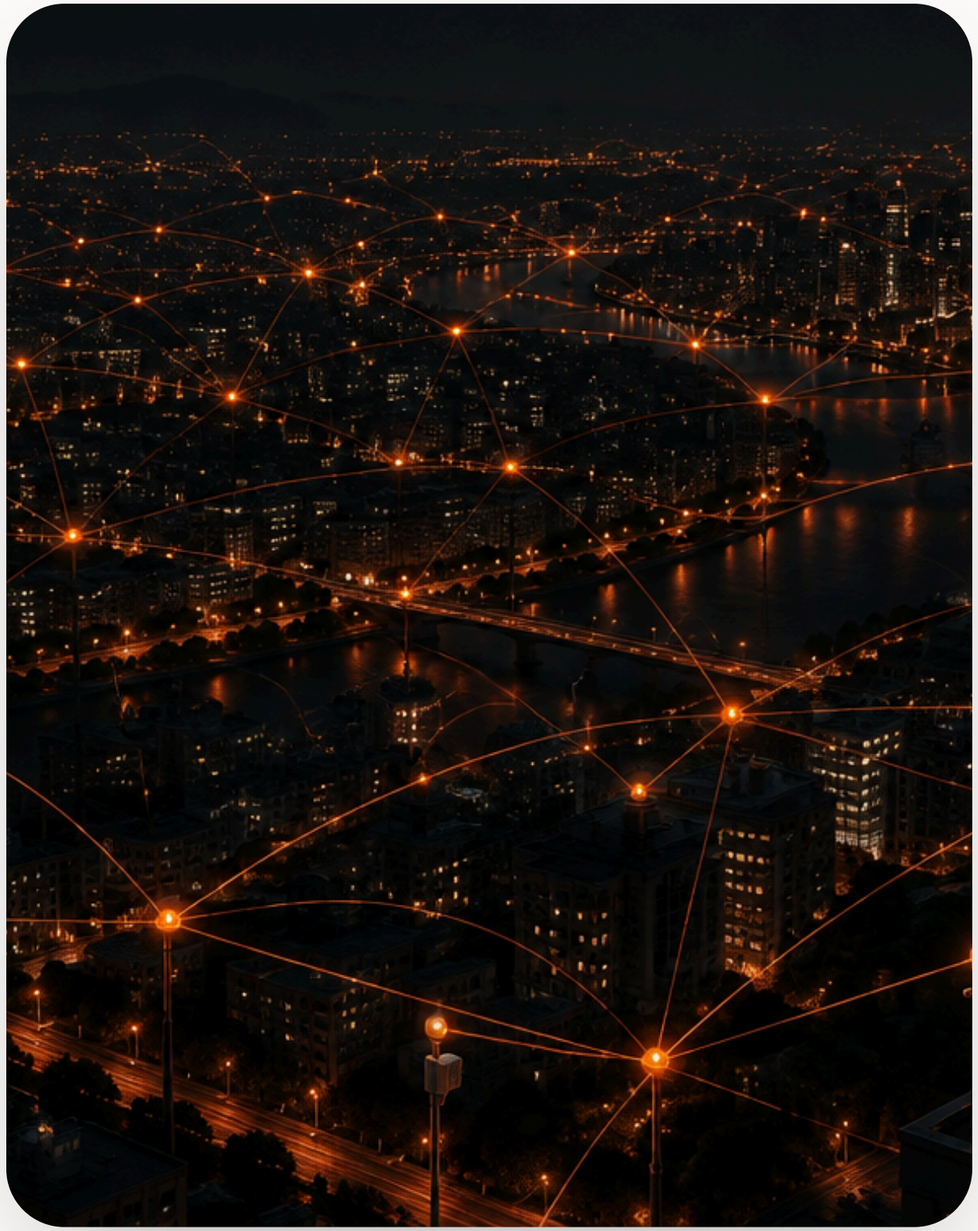
04
Bénéfices & applications
Mohamed

05
Limites & perspectives
Mohamed

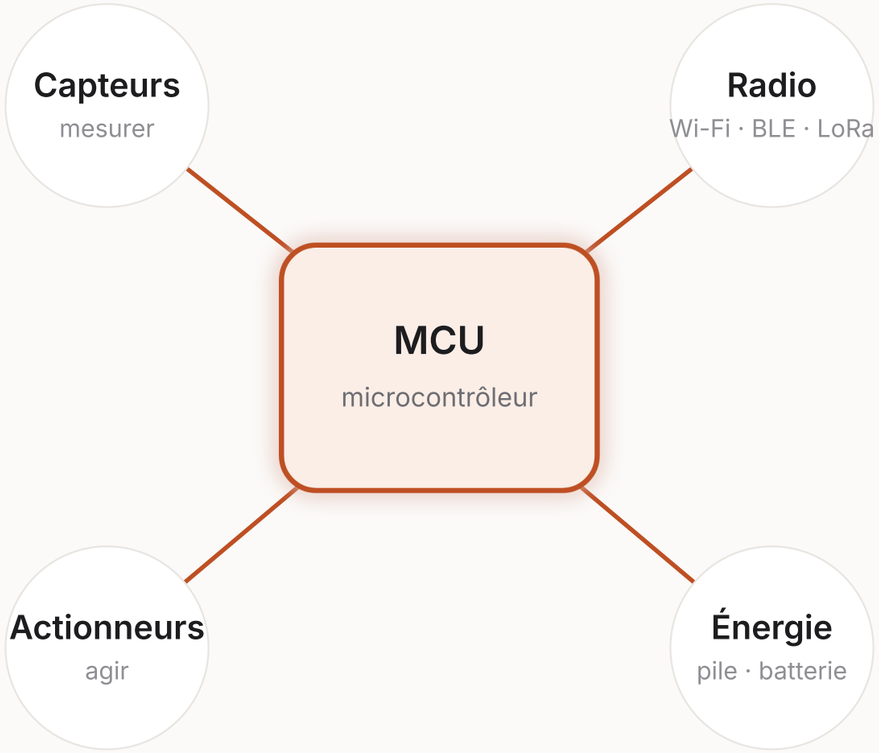
PARTIE 1

De l'IoT à l'AIoT et au TinyML

HB Hamadou BA



Anatomie d'un objet connecté



Un objet connecté **mesure** le monde physique, le **traite** sur un petit microcontrôleur et **communique** sur le réseau.

Exemple : ESP32 (Espressif)

240	520	~5 €
MHz	Ko SRAM	l'unité

Contrainte n°1 : **le budget énergétique** : des mois, voire des années sur pile.

Un système IoT en 4 couches

4	**Application**	Tableaux de bord, applis mobiles, alertes, maintenance
3	**Traitement**	Cloud : stockage, analyse… **et l'IA, aujourd'hui**
2	**Réseau**	Wi-Fi, BLE, LoRaWAN, NB-IoT · MQTT, CoAP
1	**Perception**	Capteurs, actionneurs, microcontrôleurs



Un seul capteur de vibrations…

accéléromètre · 3 axes · 1 kHz · 16 bits

518 Mo

de données brutes **par jour** à transmettre



Bande passante saturée

LoRaWAN / NB-IoT : quelques kbit/s

Latence variable

Aller-retour serveur : jusqu'à plusieurs centaines de ms

Énergie

La radio consomme plus que le calcul

Confidentialité

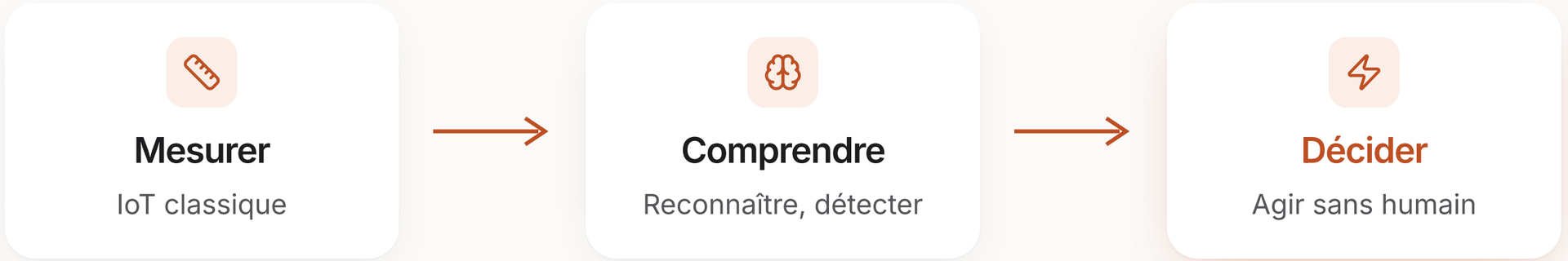
Son, santé, présence envoyés en continu

Disponibilité

Sans réseau, l'objet devient « aveugle »

AI + IoT = AIoT

Donner aux objets la capacité d'**interpréter** leurs données et de **décider**.



Une règle fixe suffit pour « T > 80 °C ». Pas pour **reconnaître un mot** ou **distinguer un roulement usé** : là, un modèle **apprend** à partir d'exemples.

1.5 · EXEMPLES

L'AIoT, partout

Industrie

Vibrations moteurs → détection d'anomalies, maintenance prédictive

Maison

Son, présence → mots-clés, automatisation

Agriculture

Humidité, images de feuilles → irrigation, maladies

Santé

Rythme cardiaque, mouvement → chutes, arythmies

Transport

Accélérations, moteur → surveillance du véhicule

Environnement

Sons de forêt → détection de tronçonneuses

Point commun : **« anomalie »** pèse quelques octets… les données brutes, des mégaoctets.

Le TinyML : l'IA à l'échelle du milliwatt

Exécuter des modèles de machine learning sur des **microcontrôleurs** à quelques euros, alimentés sur pile.

Mémoire vive disponible (échelle logarithmique)



Puissance
mW vs centaines de W

Coût
1–10 € vs milliers

Écart de mémoire
× 100 000

4 contraintes, 1 règle d'or

Mémoire

Flash : poids · SRAM : activations

Calcul

Dizaines à centaines de MHz

Énergie

Milliwatts, veille la plupart du temps

Logiciel

Pas d'OS, C/C++, pas de Python

PC / CLOUD

Entraînement

Ajuster des millions de paramètres sur des milliers d'exemples. Lourd, une seule fois.



MICROCONTRÔLEUR

Inférence

Appliquer le modèle à chaque nouvelle mesure. Léger, en continu.

1.8 · FONCTIONNEMENT

La boucle capteur → action



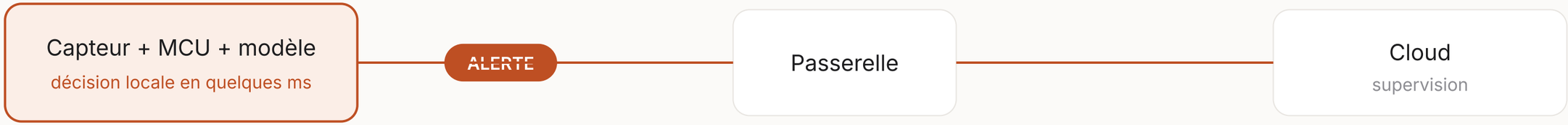

```
void loop() {
  lire_accelerometre(fenetre, 1000);          // 1. acquisition (1 s à 1 kHz)
  calculer_caracteristiques(fenetre, entree);    // 2. prétraitement
  interpreter->Invoke();                          // 3. inférence locale (TFLM)
  if (sortie[0] > SEUIL_ANOMALIE)                // 4. décision
    mqtt.publish("usine/moteur1/alerte", "anomalie");  // 5. seule l'alerte part
  esp_light_sleep_start();                        // veille jusqu'à la prochaine fenêtre
}
```

Cloud vs TinyML

Architecture cloud : on envoie tout



Architecture TinyML : on décide sur place



1.10 · SYNTHÈSE

Ce qui change vraiment

CRITÈRE	CLOUD	TINYML
Données envoyées	Brutes, en continu	Alertes uniquement
Latence	Dépend du réseau	Quelques ms, locale
Hors ligne	Impossible	Possible
Confidentialité	Données à distance	Données dans l'objet
Taille des modèles	Illimitée	Quelques Ko

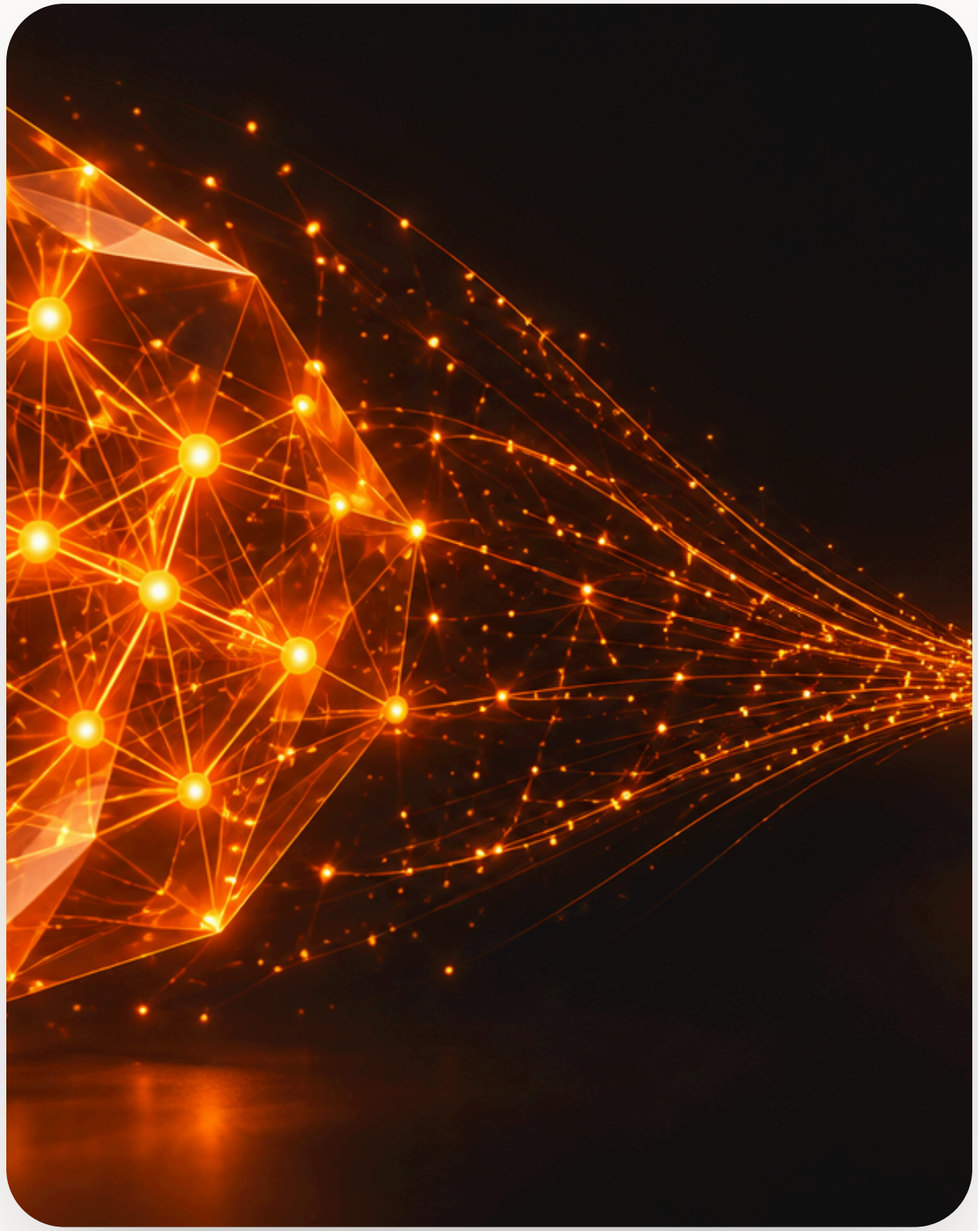
Exemple illustratif

Détecter les vibrations anormales d'un moteur avec un ESP32 + accéléromètre

PARTIES 2 & 3

Principes techniques, matériel & outils

NC Nicolas CHEVALIER



2.1 · PIPELINE

Du laboratoire au microcontrôleur

1

Données

Collecte sur le capteur réel. La plus grande part du temps.

2

Entraînement

TensorFlow / PyTorch sur PC. Float32, plusieurs Mo.

3

Optimisation

Quantification, élagage, distillation → `.tflite`

4

Firmware

Modèle = tableau d'octets en mémoire flash

5

Inférence

Sur le MCU, sans réseau

⚠ Le microcontrôleur **n'apprend pas** : le modèle est **figé à la compilation**.

2.2 · QUANTIFICATION

Float32 → Int8 : la taille ÷ 4

$$r \approx S \times (q - Z)$$

S = 1,52 / 255 ≈ 0,00596 · Z = −128 − round(min / S) = 18

57	−57	38	−128	127	−37

Taille
÷ 4

Erreur max / poids
< 0,003

Énergie d'une addition
÷ 30
Horowitz 2014

Perte de précision
~1–2 pts

PTQ ou QAT ?

PTQ

Post-entraînement

On convertit un modèle déjà entraîné. Rapide, simple, perte < 1–2 %.

QAT

Pendant l'entraînement

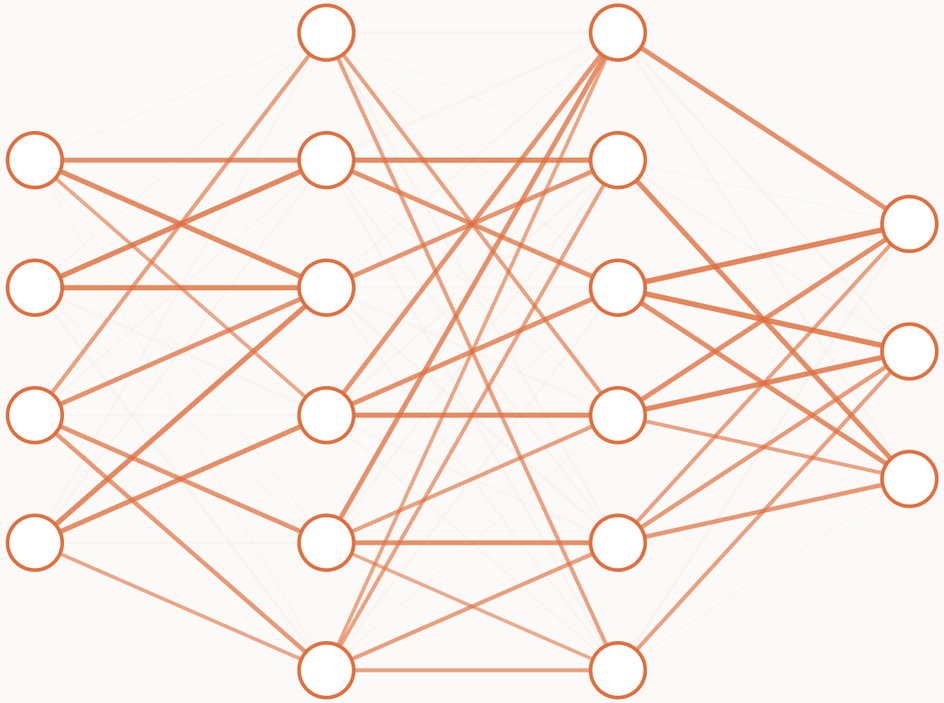
« Fake quantization » : le modèle s'habitue à l'Int8. Permet Int4, binaire.

```python
# Quantification post-entraînement Int8
import tensorflow as tf

conv = tf.lite.TFLiteConverter.from_saved_model(dossier)
conv.optimizations = [tf.lite.Optimize.DEFAULT]
# données représentatives pour calibrer S et Z
conv.representative_dataset = donnees_repr
conv.target_spec.supported_ops = [
    tf.lite.OpsSet.TFLITE_BUILTINS_INT8]
conv.inference_input_type  = tf.int8
conv.inference_output_type = tf.int8

modele = conv.convert()
open("modele_int8.tflite", "wb").write(modele)
```

Couper les connexions inutiles



Beaucoup de poids sont **proches de zéro** : ils pèsent peu dans la décision.

Non structuré

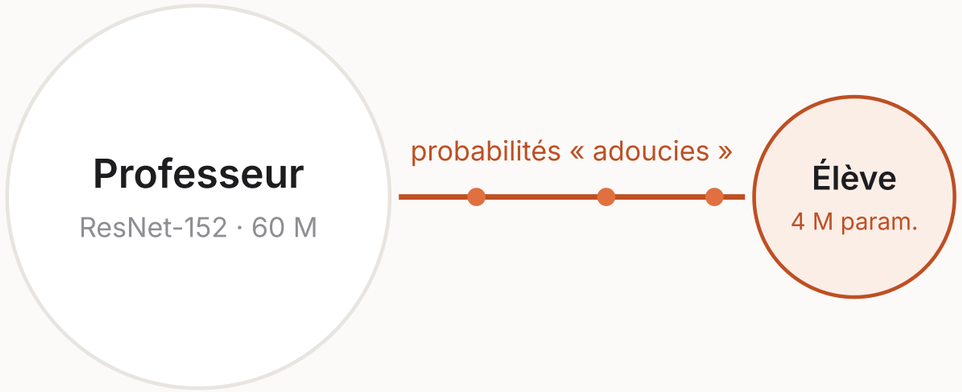
Poids un par un · jusqu'à 90 % de compression · formats creux nécessaires

Structuré

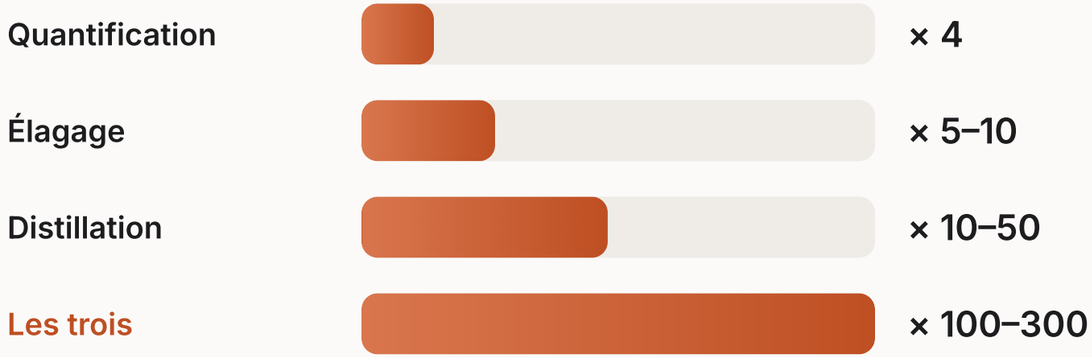
Neurones, filtres entiers · gain réel en calcul · compatible matériel

Stratégie : **20 % par itération** + fine-tuning, plutôt qu'un seul gros coup.

Le professeur, l'élève… et le combo



Réduction de taille (ordres de grandeur)



$$P_i = \exp(z_i/T) \,/\, \Sigma \exp(z_j/T)$$

Des puces de plus en plus intelligentes

Espressif

ESP32-S3

Cœur	Xtensa LX7 · 240 MHz
SRAM	512 Ko
IA	SIMD 8/16 bits
Radio	Wi-Fi · BLE
Fréquence	

Espressif

ESP32-P4

Cœur	RISC-V · 400 MHz
PSRAM	jusqu'à 32 Mo
IA	Extensions IA
Sortie	2024
Fréquence	

STMicroelectronics

STM32N6

Cœur	Cortex-M55 · 800 MHz
NPU	Neural-ART · 600 GOPS
Efficacité	~3 TOPS/W
SRAM	4,2 Mo
Fréquence	

Pour surveiller un moteur : un **ESP32** suffit largement, et son Wi-Fi envoie les alertes.

La chaîne logicielle

Entraînement
TensorFlow · PyTorch

→

Conversion
TFLite Converter

→

Exécution
TFLM · ESP-DL

→

Firmware
ESP-IDF · CubeIDE

TensorFlow Lite Micro

Interpréteur C++ sans OS ni allocation dynamique · quelques dizaines de Ko · `tensor arena` statique

Edge Impulse

Plateforme en ligne : collecte, FFT/MFCC, entraînement, export C++ prêt à compiler

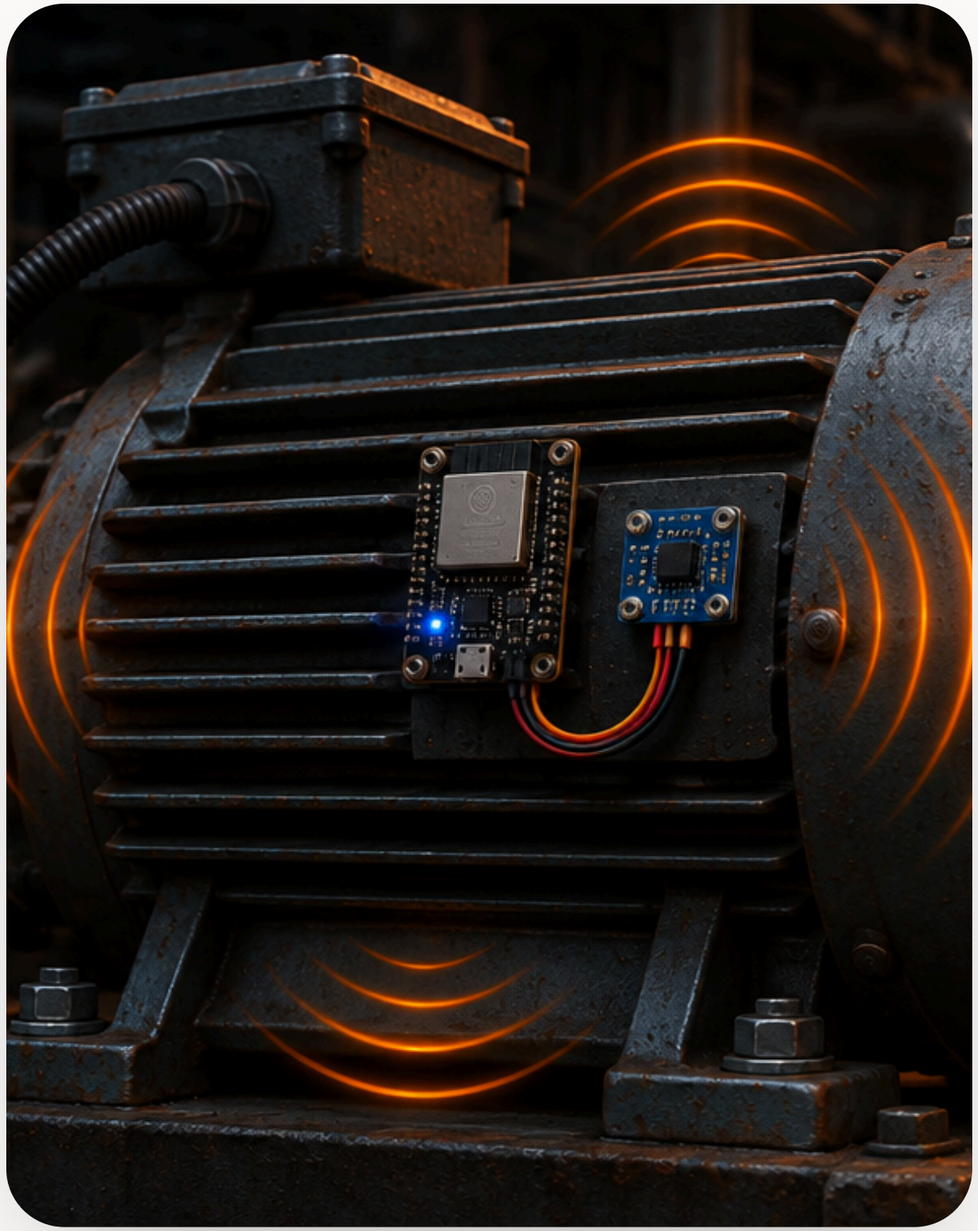
Outils constructeurs

ESP-DL (Espressif) · STM32Cube.AI (ST) · eIQ (NXP)

PARTIES 4 & 5

Bénéfices, applications & limites

MS Mohamed SANOGO



4.1 · BÉNÉFICES

Les promesses… et leurs nuances

Réactivité

Pas d'aller-retour réseau : décision en quelques ms.

Confidentialité

Les données brutes restent dans l'objet ; on envoie une alerte.

Énergie

Moins de radio = plus d'autonomie.

Hors ligne

L'analyse continue sans Internet.

Les promesses… et leurs nuances

MAIS

Pas de « latence zéro »

Acquisition, prétraitement et inférence prennent toujours du temps.

MAIS

Pas automatique

Le système peut encore transmettre pour l'entraînement ou le suivi : c'est un choix.

MAIS

Le calcul consomme aussi

Le gain se mesure sur le dispositif complet, pas sur le modèle seul.

MAIS

Si c'est prévu

Seulement pour les fonctions conçues pour tourner en autonomie.

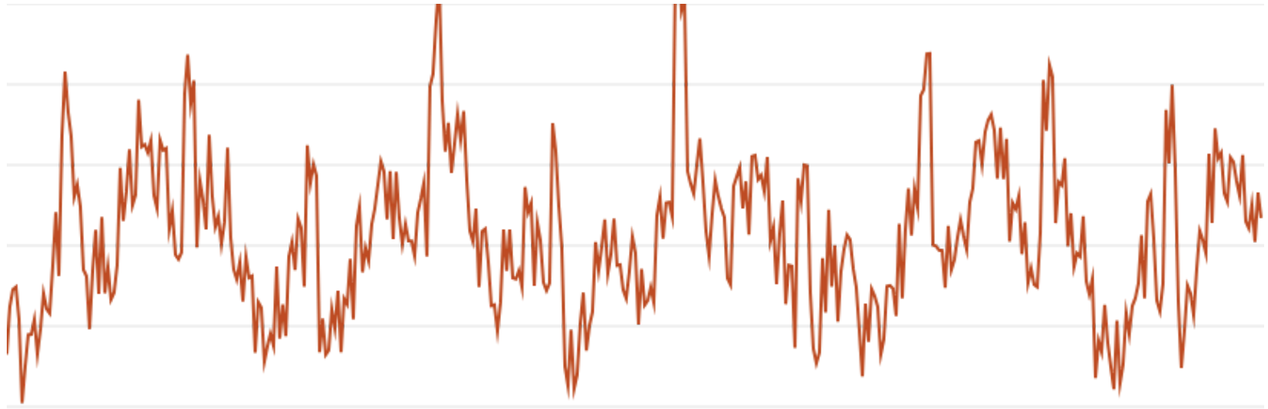
4.2 · INDUSTRIE · SCÉNARIO ILLUSTRATIF

Simulation : un moteur sous surveillance

Signal simulé, à titre d'illustration

accéléromètre · axe Z

● ANOMALIE



score d'anomalie



0.92

MQTT · usine/moteur1/alerte

« anomalie détectée » : **quelques octets** envoyés au lieu de 500 Mo/jour

Injection d'un défaut (balourd, roulement usé)

À RETENIR

Détecter une **anomalie** ≠ prédire une **panne**. C'est un premier niveau de surveillance.

« Lumière ! » : les mots-clés



« LUMIÈRE » ✓

Tâche ciblée

Quelques commandes, un modèle de quelques dizaines de Ko. Référence du banc d'essai **MLPerf Tiny**.

Le son reste local

Seule la commande reconnue est transmise, pas l'enregistrement.

Le piège : le bruit

Entraîner et tester avec des données représentatives des vraies conditions.

5.1 · LIMITES & DÉFIS

Ce qui reste difficile

Ressources limitées

Compromis permanent taille / précision / temps / énergie. TFLM : opérations limitées.

Fiabilité & données

Bruit, utilisateurs, position du capteur : un modèle bon en test peut dériver sur le terrain.

Faire évoluer le modèle

Entraînement hors de l'objet → mise à jour OTA. L'apprentissage sur l'appareil progresse (< 256 Ko), sans être généralisé.

Sécurité

Extraction ou modification du modèle, entrées piégées → flash chiffrée, démarrage sécurisé, OTA signée.

Vers une intelligence distribuée



Le TinyML ne **remplace** pas le cloud : il le **complète**. Chaque niveau fait ce qu'il fait le mieux.

CONCLUSION

En trois idées

1

Du capteur qui mesure à l'objet qui décide

IoT → AIoT → TinyML : l'IA descend jusqu'au microcontrôleur.

2

Entraîner ailleurs, inférer sur place

La compression réduit un modèle de 100 à 300 fois.

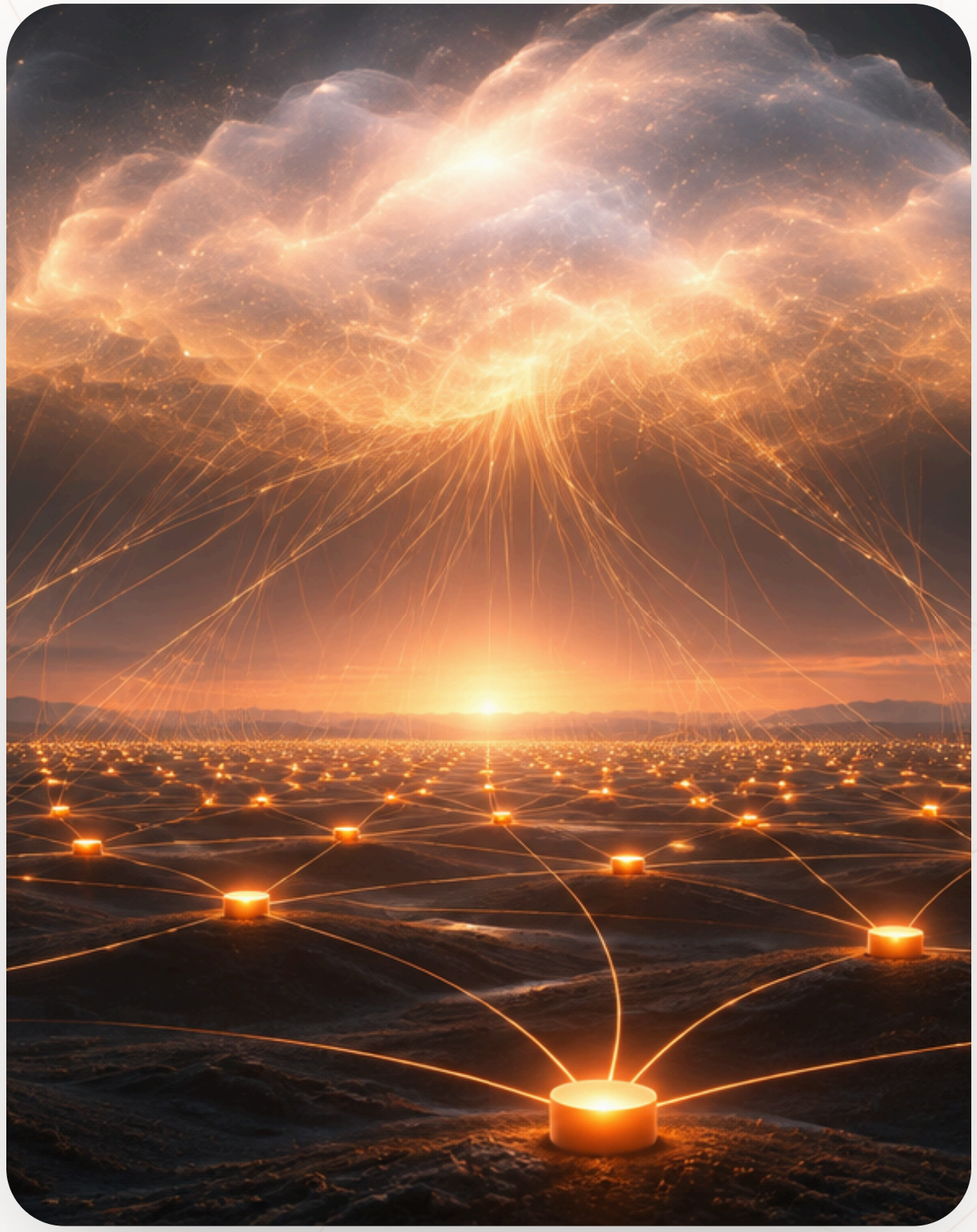
3

Des gains réels, mais nuancés

Réactivité, confidentialité, énergie… dans une architecture **hybride**.

Merci.

Des questions ?



Hamadou BA · Nicolas CHEVALIER · Mohamed SANOGO

Encadrant : M. Claude DUVALLET · Université Le Havre Normandie